Reinforcement Learning for Adaptive Noise-Aware Quantum Compilation

Ryan Luedtke

Northwestern University

Evanston, Illinois, USA

ryanluedtke2026@u.northwestern.edu

Abstract

Noisy intermediate-scale quantum (NISQ) processors exhibit significant hardware variability and time-dependent error behavior that strongly impacts circuit reliability. This heterogeneity in qubit quality, gate fidelity, and readout reliability makes quantum compilation decisions a key determinant of end-to-end circuit performance. Prior work has shown that noise-aware qubit mapping can substantially improve success probability by prioritizing higher-quality physical regions of a device when mapping logical qubits. However, most existing approaches take a static approach, optimizing layouts using a calibration snapshot or manually-defined policy. These layouts do not improve through repeated interaction with hardware feedback.

This paper explores reinforcement learning as a framework for adaptive noise-aware quantum compilation. I first implement a baseline experimental pipeline that compares multiple mapping strategies on simulated IBM backends using known-output benchmark circuits. The system extracts backend error data, evaluates mappings using a noise-aware reliability score derived from backend calibration information, transpiles circuits under fixed layouts, and measures empirical success probability under simulated noise. This preliminary experiment demonstrates that mapping choices alone materially influence observed circuit success rates, supporting the claim that compilation is a meaningful optimization target for NISQ processors.

Using this experiment as a baseline, I then propose a full reinforcement-learning (RL) pipeline in which compilation is treated as a sequential decision problem. The proposed `RLCompiler` environment models layout construction as a Gymnasium [13] environment with discretized actions, hardware-aware observations, and reward signals derived from both analytic proxy metrics and empirical execution outcomes. This staged RL pipeline, which combines reliability-based proxy rewards derived from backend calibration data, canary circuits with known outputs, and eventually runtime characterization signals offers a practical path toward an adaptive compiler that learns policies rather than relying exclusively on static heuristics.

CCS Concepts

• **Hardware** → **Quantum computation**; • **Software and its engineering** → **Compilers**; • **Computing methodologies** → **Reinforcement learning**.

Keywords

quantum compilation, NISQ, reinforcement learning, noise-aware mapping, Qiskit, PPO

1 Introduction

Today's quantum computers operate in the noisy intermediate-scale quantum (NISQ) era [5], where hardware noise, short coherence times, and sparse coupling constraints strongly shape what circuits can be executed reliably. Rather than treating compilation as a trivial translation step, NISQ systems require compilation to act as a noise-sensitive optimization layer. Mapping a logical circuit onto different physical qubits can change the number of inserted SWAP gates, the quality of the two-qubit links used during routing, and the readout fidelity of final measurements[6]. As a result, compilation decisions have a significant and direct effect on execution quality.

This sensitivity has already motivated noise-aware compilation techniques. Murali et al. show that backend-aware mappings which explicitly account for calibration data can significantly improve execution reliability relative to baseline compiler workflows [1]. Tannu and Qureshi similarly argue that NISQ processors exhibit meaningful qubit-to-qubit and link-to-link variability, and that compilers should account for this nonuniformity when selecting mappings and movement policies [2]. Together, these works establish a key motivation for informed quantum compilation: not all physical qubits are equally useful, and intelligent compilation should exploit that fact.

While advancements have been made in noise-aware quantum compilation, existing approaches are still largely static. They optimize against a fixed calibration snapshot, or they encode their domain knowledge as manually-determined heuristics. In practice, device noise is stochastic, spatially heterogeneous, and often time-varying. A mapping policy that is optimal for one circuit or one hardware state may be suboptimal later. This limitation suggests a natural alternative: treat compilation as a sequential decision problem and learn compilation behavior from repeated experience.

The central idea of this report is to investigate reinforcement learning as a mechanism for adaptive noise-aware quantum compilation. To explore this direction, I first develop an experimental framework for evaluating hardware-aware qubit mappings on simulated NISQ backends. The system extracts backend calibration data, enumerates and scores candidate layouts, transpiles circuits under fixed

mappings, and measures empirical success probability under noisy simulation. These experiments provide direct evidence that mapping choice alone can significantly influence execution reliability on realistic device models.

Building on this framework, I then describe a proposed RLCompiler pipeline that would treat layout construction as a sequential decision problem. The environment exposes hardware noise information, circuit structure, and partial layout assignments to an agent that learns mapping policies through interaction with simulated backends. I outline how such an environment could be trained using reinforcement learning algorithms and explain how the experimental infrastructure developed here naturally supports this transition from heuristic evaluation to learned compilation strategies.

The main contributions of this report are:

- a preliminary experimental framework for evaluating noise-aware qubit mappings on realistic simulated backends,
- a Murali-inspired mapping score implemented as a practical baseline objective,
- a proposed RLCompiler environment that formalizes compilation as a sequential decision process, and
- a detailed future pipeline explaining how reinforcement learning could be trained and evaluated in practice.

2 Background and Prior Work

2.1 Noise-Aware and Variability-Aware Mapping

Murali et al. introduced a noise-adaptive compiler mapping framework for NISQ hardware that integrates device calibration data into layout selection and routing. Their key insight is that circuit reliability can be approximated by combining the reliabilities of measurement operations and two-qubit gates across a candidate mapping. By formulating compilation as a reliability optimization problem, they show that backend-aware mappings can outperform naive compilation significantly in both average and worst-case settings. This work is especially relevant here because it provides both a systems motivation and a practical scoring formulation for mapping quality.

A closely related perspective appears in the variability-aware work of Tannu and Qureshi. They argue that NISQ devices are not homogeneous computational substrates. Instead, qubits and couplings exhibit varying error rates, and these differences should influence how programs are allocated and moved across the device. Their proposed policies emphasize that compilers should avoid weak regions of the hardware when possible, rather than assuming all physical qubits are interchangeable. This observation strongly motivates learning-based approaches, since a learned policy may be able to exploit structured variability that is difficult to encode with static rules alone.

2.2 Canary Circuits and Runtime Characterization

A practical challenge in learning-based quantum compilation is reward construction. For many interesting quantum programs, the correct output distribution is not known classically. This makes it difficult to use direct success probability as a reward signal during training. One useful idea is to rely on *canary circuits*, which are structurally informative circuits with known outputs that can serve as probes for compilation quality. The recent work on Clifford canary circuits proposes exactly this type of strategy, using canaries as a way to infer which execution conditions are likely to yield higher fidelity [3]. This idea is directly relevant to my project because my baseline experiments already rely on simple known-output circuits such as Bell and GHZ states and small arithmetic circuits with deterministically checkable outcomes.

Another relevant line of work is dynamic resource allocation with quantum error detection. The recent Pauli-check-based characterization and allocation papers suggest that runtime signals, including detected syndromes and post-selection yield, can be used to characterize noise and guide better resource usage [4]. Although my current implementation does not yet incorporate these signals, they offer a compelling future mechanism for extending an RL reward function beyond simple known-output benchmarks. In the longer term, such signals could allow an RL-based compiler to learn even when application-level outputs are unknown.

2.3 Reinforcement Learning in Quantum Compilation

Reinforcement learning has also begun to appear in quantum compiling and related optimization settings. Recent work has explored deep reinforcement learning for quantum compiling and has demonstrated RL-based compilation on superconducting hardware [7, 8]. At the same time, most prior work on NISQ qubit mapping has still relied primarily on heuristic or analytically derived policies rather than learned ones [1, 2]. These approaches demonstrate that mapping and scheduling decisions significantly impact circuit fidelity, but they do not adapt policies through interaction with hardware or simulated environments.

The appeal of RL in this setting is straightforward. Compilation decisions are sequential, action choices affect future opportunities, and the final quality of a sequence may only be observable after the entire decision process is complete. This aligns naturally with the sequential decision-making framework formalized in reinforcement learning [11].

3 Proposed Approach

The proposed research direction is a reinforcement-learning-based compiler that learns adaptive qubit mapping policies from circuit structure and backend noise information. Instead of selecting a mapping using a fixed hand-designed heuristic, the compiler would

treat layout construction as a sequential decision process and optimize a policy over repeated interaction with simulated or real hardware feedback.

The proposed system has two stages. The first stage is the current baseline experiment, which compares several mapping strategies using realistic backend noise data and noisy simulation. This baseline establishes whether mapping choice measurably affects performance and whether heuristic signals such as Murali-style reliability scores correlate with empirical success. The second stage is a full RL pipeline in which a policy is trained to choose mappings directly.

The intuition behind this approach is that static mapping heuristics are limited by the assumptions built into their objective functions. A learned policy, by contrast, can exploit richer interactions among topology, circuit structure, and observed performance. In principle, it can learn when a lower-readout-error qubit is worth a longer path, when stronger connectivity matters more than local readout quality, or when particular benchmark structures benefit from specific hardware regions. This does not mean reinforcement learning replaces compiler theory. Rather, it augments classical compiler logic with a data-driven policy layer.

4 Methods and System Design

4.1 Baseline Experimental Framework

The implementation developed for this project is built in Python using Qiskit [12] for circuit generation, transpilation, fake backend snapshots, and noisy simulation. Qiskit is the primary systems framework used to interface with circuit-level compilation and IBM-style backend models. The current codebase focuses on a controlled noise-mapping experiment rather than a full RL training loop, but it already includes several components that naturally support the transition to reinforcement learning. The implementation supports multiple IBM-style fake backends, including FakeManilaV2, FakeLimaV2, and related five-qubit topologies, although the results shown here focus on a representative backend for clarity.

The experimental framework generates known-output benchmark circuits, extracts backend error information, evaluates multiple candidate mappings, transpiles each circuit under a chosen initial layout, and runs the resulting circuit on a simulator configured from the backend. The final metric is empirical success probability, computed as the fraction of output shots that match the known valid outcome set for the benchmark.

The implementation for this project, including the noise-mapping experiment pipeline, visualization utilities, and RLCompiler scaffold, is available in the accompanying code repository [14].

4.2 Benchmark Circuits

The baseline uses several small circuits whose correct outputs are known in advance:

- a Bell-state preparation circuit with valid outcomes $\{00, 11\}$,
- a GHZ-state circuit with valid outcomes $\{000, 111\}$, and
- a small ripple-carry adder benchmark whose expected sum bitstring can be computed classically.

These are intentionally simple. Their role is not to demonstrate quantum advantage, but to serve as clean probes for layout sensitivity. Because the output is known exactly, they provide a stable metric for evaluating mapping quality and later offer a straightforward reward mechanism for early-stage RL training.

4.3 Backend Noise Extraction

A compact backend representation is extracted from each fake backend snapshot. The information used in the baseline includes:

- per-qubit readout error,
- per-edge two-qubit gate error, and
- hardware coupling edges defining the physical connectivity graph.

These values are sufficient to support both the heuristic mapping score and the visualization tools developed for the experiments. They also form a natural subset of what an RL environment should expose in its state representation.

4.4 Murali-Inspired Mapping Score

A Murali-inspired score is used to rank candidate mappings. The implementation follows the structure of a log-reliability objective. Each candidate mapping receives a score based on two terms:

(1) the readout reliability of the mapped physical qubits, and
(2) the reliability of the best physical path between qubits that interact through logical CNOT edges in the circuit.

Path reliability is estimated by treating the coupling graph as a weighted graph whose edge weights are derived from two-qubit gate success probabilities. The most reliable path between two physical qubits is then found using Dijkstra-style search on negative log-reliability weights. This yields an additive surrogate for multiplicative path success probability.

This scoring function is useful for two reasons. First, it gives a principled noise-aware baseline grounded in prior compiler work. Second, it is computationally cheap enough to be used later as a shaped reward or proxy reward during RL training.

More formally, for a candidate layout L, the score is defined as

$$\mathrm{score}(L) = \omega \sum_{q \in L} \log\left(1-\epsilon_q^{\mathrm{read}}\right) + (1-\omega) \sum_{(u,v) \in E_{\mathrm{CX}}} \log\left(R_{\mathrm{best}}(L(u), L(v))\right),$$

where $\epsilon_q^{\mathrm{read}}$ is the readout error of physical qubit q, and $R_{\mathrm{best}}(i, j)$ is the reliability of the most reliable path between physical qubits i and j.

The best-path reliability is computed as

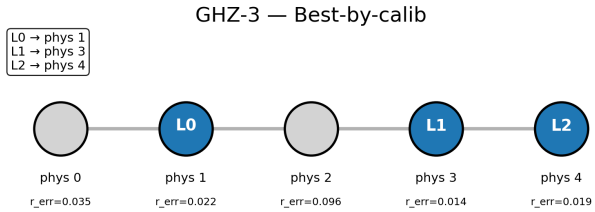

Figure 1: Example visualization of a candidate logical-to-physical mapping on a simulated backend. Shown is the GHZ mapping on FakeManila using best-by-calibration mapping.

$$R_{\text{best}}(i,j) = \max_{p \in \mathcal{P}(i,j)} \prod_{(a,b)\in p} (1 - \epsilon^{2q}_{ab}),$$

where ϵ^{2q}_{ab} is the two-qubit gate error on edge (a,b) and $\mathcal{P}(i,j)$ denotes all paths between i and j. In practice, this is computed using Dijkstra's algorithm on negative log-reliability weights.

4.5 Mapping Strategies Compared

The baseline experiment compares several strategies:

- **Random mapping**, which serves as a control baseline and is averaged over multiple trials.
- **Best-by-calibration**, which selects the candidate layout with the highest Murali-inspired reliability score.
- **Worst-by-calibration**, which selects the candidate layout with the lowest Murali-inspired reliability score as a sanity check.
- **A Tannu-VQA-like heuristic**, which prioritizes physical qubits with stronger local connectivity and lower incident error.

Because the benchmarks are small and the backends have only five physical qubits, layouts can be enumerated exhaustively for the baseline experiments. This allows direct comparison among a complete set of candidate mappings.

4.6 Visualization

As shown in Figure 1, the current implementation includes a hardware-topology visualization component that plots the backend coupling graph, highlights selected physical qubits, and annotates them with logical labels and readout errors. These visualizations are especially useful for interpreting why a mapping succeeded or failed, since they make the relationship between topology, qubit quality, and logical assignment immediately visible.

5 Preliminary Results

The most important outcome of the preliminary experiments is not any single absolute success value, but the fact that *mapping choice alone* changes measured performance in a meaningful way. Success probability is computed as

$$\text{SuccessProbability} = \frac{\sum_{b \in O_{\text{valid}}} \text{counts}(b)}{\sum_b \text{counts}(b)},$$

where O_{valid} is the set of correct output bitstrings for the benchmark circuit. When the same benchmark circuit is transpiled under different fixed layouts on the same noisy backend, empirical success probability varies across mappings. This confirms the core premise of the project: qubit mapping is a meaningful optimization problem in the NISQ setting.

The calibration-aware and connectivity-aware strategies capture meaningful structure in the hardware, but their empirical performance is not perfectly stable across runs or uniformly ordered across all benchmarks. In repeated experiments, Bell circuits consistently achieve the highest success probability, followed by GHZ and then the adder benchmark, indicating that circuit difficulty is a dominant source of variation. However, the relative performance of individual mapping strategies can change from run to run, especially on the more complex benchmarks. In the runs examined here, best-by-calibration typically performs strongest on Bell and GHZ, while the Tannu-VQA-like heuristic often performs better on the adder benchmark. This suggests that while heuristic scores capture useful information about backend quality, they do not fully determine end-to-end success under noisy execution. Rather than weakening the motivation for this work, this variability strengthens the case for adaptive methods that learn from repeated execution feedback instead of relying exclusively on static mapping rules.

For example, in one run on the simulated backend, the Bell benchmark achieved success probabilities of 0.911 for random mapping, 0.965 for the best-by-calibration layout, 0.934 for the worst-by-calibration layout, and 0.885 for the Tannu-VQA-like heuristic. On GHZ-3, the corresponding values were 0.849, 0.868, 0.848 and 0.859. On the adder benchmark, the values were lower overall and less consistently aligned with the calibration-based ranking: 0.644 for random mapping, 0.646 for best-by-calibration, 0.663 for worst-by-calibration, and 0.709 for the Tannu-VQA-like heuristic. These results reinforce two observations. First, benchmark difficulty is clearly structured, with Bell performing best, followed by GHZ and then the adder. Second, the ranking among mapping strategies is not completely fixed, especially on more complex circuits, which indicates that static proxy objectives do not fully capture realized execution behavior.

The current baseline also demonstrates that visualization is not merely cosmetic. In several cases, plots of the selected mappings show why two strategies differ. One mapping may place entangled logical qubits on adjacent physical qubits with low readout error, while another places them on weaker or more distant qubits and thereby induces more fragile execution. These are exactly the kinds of relationships a future RL agent should learn to identify without requiring an explicit hand-designed rule for every case.

These results are preliminary in the strongest sense: they are not yet RL results. However, they are sufficient to establish viability. If mapping choice did not affect performance, then a learned compiler would have little to optimize. The baseline shows the opposite.

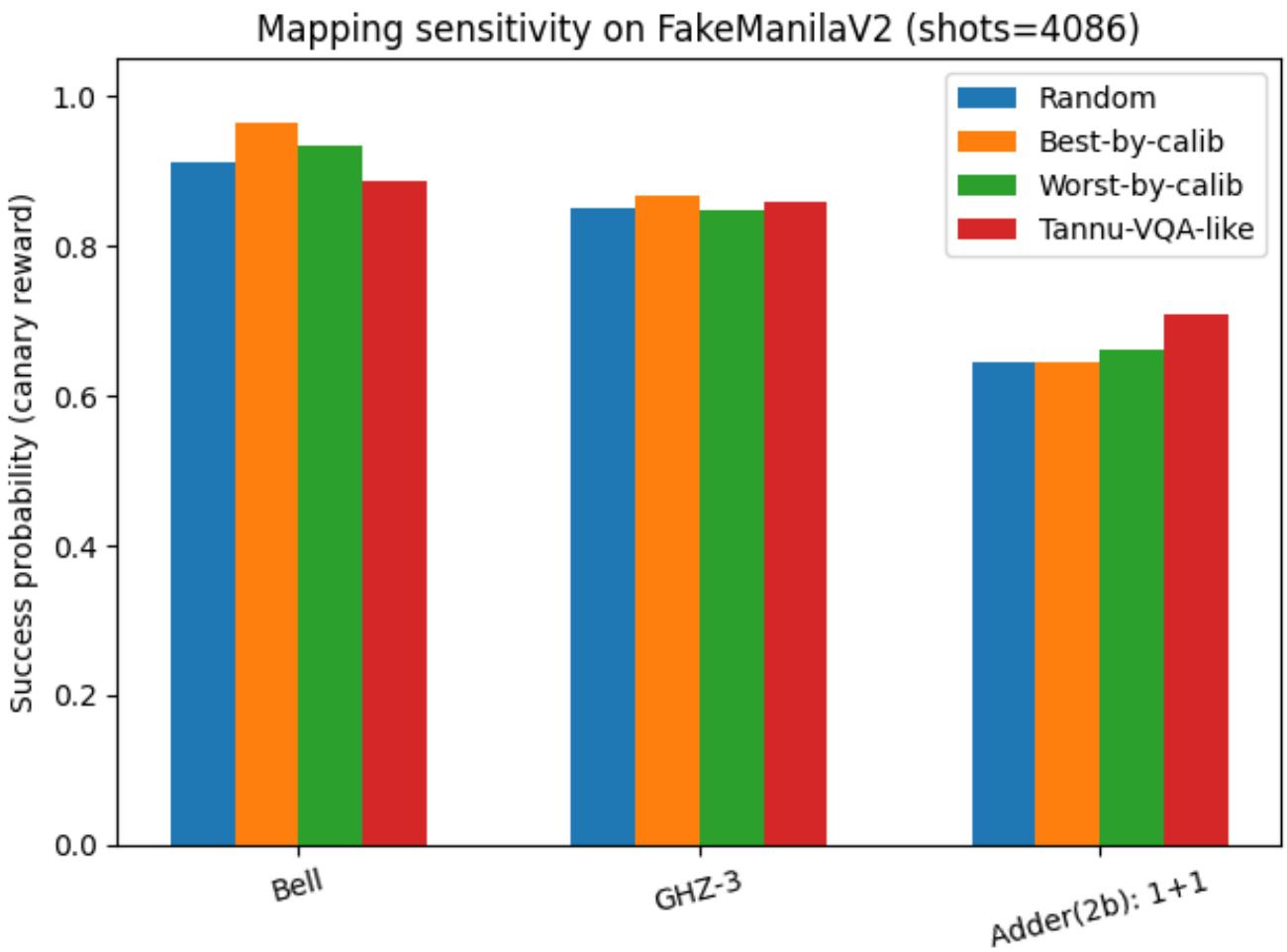

Figure 2: Comparison of empirical success probability across mapping strategies on a simulated backend. Results are shown for Bell, GHZ, and small adder benchmarks. The variation across strategies highlights the sensitivity of execution quality to mapping decisions and demonstrates that proxy reliability metrics do not uniformly predict empirical success across all circuits.

There is measurable structure in the problem, and that structure appears to align at least partially with noise-aware heuristics derived from prior work.

6 Future RL Pipeline

6.1 RLCompiler as a Sequential Environment

The natural next step is to formalize qubit mapping as a reinforcement learning environment. In the proposed RLCompiler design, each episode corresponds to compiling one circuit for one backend snapshot. The agent constructs a mapping sequentially by assigning one logical qubit at a time to an unused physical qubit. An overview of this pipeline is shown in Figure 3.

A convenient Markov decision process formulation is:

- **State:** partial logical-to-physical mapping, mask of used physical qubits, backend noise features, and eventually circuit interaction features.
- **Action:** select a physical qubit for the next logical qubit.
- **Transition:** update the partial layout and advance to the next decision step.
- **Reward:** initially sparse and terminal, later augmented with shaped terms.

This structure is particularly compatible with PPO [9] because the action space is discrete and the horizon is short for small circuits. The implementation is designed to integrate with standard RL libraries such as Stable-Baselines3 [10] and Gymnasium [13]. PPO is also a reasonable first algorithm because it is robust, widely used, and already well supported in Stable-Baselines3.

6.2 Why a Staged Reward Pipeline Makes Sense

A full RL compiler should not begin training directly on expensive end-to-end empirical rewards alone. Instead, a staged reward pipeline is more practical.

Stage 1: Proxy reward. The first stage uses the Murali-inspired score as a terminal reward. This is computationally cheap, deterministic given a backend snapshot, and directly tied to prior noise-aware compiler objectives. It allows rapid experimentation with state and action design before simulator noise is introduced into the learning loop.

Stage 2: Canary reward. Once the environment is stable, training can incorporate empirical rewards from canary circuits with known outputs. Because Bell, GHZ, and arithmetic canaries have classically checkable outcomes, they provide a much stronger signal than an analytic proxy alone. This stage allows the agent to learn from actual transpilation and noisy execution rather than only from a hand-designed score.

Stage 3: Hybrid reward. A hybrid reward can combine proxy and empirical terms. This is attractive because it gives the agent both a dense structural signal and an end-to-end correctness signal. For example, a reward may combine Murali reliability, success probability on a simulator, and penalties for excessive depth or SWAP count.

Stage 4: Runtime characterization signals. For more complex application circuits whose outputs are not known, canary-style probes or Pauli-check-based characterization signals could be used to provide reward or auxiliary state information. This would extend the RL compiler beyond toy benchmarks and toward realistic NISQ workloads.

This staged approach is promising in practice because it addresses distinct bottlenecks at each step. The proxy stage makes the learning problem tractable, while the canary stage anchors the policy in observed execution behavior. A hybrid stage can then bridge structured rewards and realistic outcomes, and runtime characterization signals offer a path toward learning even when correctness cannot be directly evaluated.

6.3 Training Data and Generalization

In this RL setting, the "training data" is not a static labeled dataset. Instead, it is generated through interaction. Episodes would be sampled over a distribution of:

- circuits,
- backend snapshots,
- benchmark families, and
- noise profiles.

This matters for generalization. A policy trained on only one circuit or one backend could easily memorize a narrow mapping preference rather than learn a reusable compiler strategy. By training across diverse benchmark circuits and multiple fake backends, the agent is encouraged to learn general principles such as preferring high-quality qubits, preserving short entangling paths, and avoiding weak hardware regions.

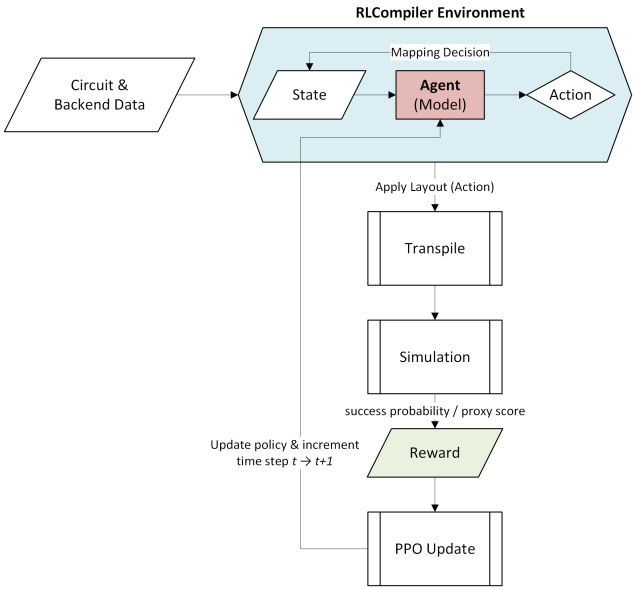

Figure 3: Proposed reinforcement-learning pipeline for adaptive noise-aware quantum compilation.

6.4 Beyond Initial Placement

The current RLCompiler concept only addresses initial layout, but that is a deliberate design choice rather than a long-term limitation. Initial placement is the cleanest place to begin because it is compact, discrete, and already has a meaningful reward signal. Once that stage is working, the same framework could be extended to:

- routing decisions,
- SWAP insertion policies,
- scheduling and timing,
- multi-objective optimization of fidelity and depth.

This progression mirrors how classical systems work often evolve: begin with one tractable optimization stage, validate its usefulness, then integrate it into a larger pipeline.

7 Discussion

The current project already supports a meaningful claim: even simple benchmark circuits exhibit measurable mapping sensitivity under realistic noisy simulation, and structured noise-aware heuristics often outperform uninformed random layouts, though the relative ordering of strategies is not perfectly stable across repeated runs or across benchmarks.

At the same time, the transition from heuristic experiments to reinforcement learning introduces nontrivial design challenges. Observation design must include enough structural information for a policy to generalize. Rewards must be informative without being prohibitively expensive. Training must avoid collapsing into benchmark-specific memorization. These are real challenges, but they are also exactly the kinds of design problems for which staged systems research is well suited.

I view the current baseline not as a separate project from the RL proposal, but as the first layer of it. The code already implements the backend-error extraction, reliability estimation, simulator-based evaluation, and visualization infrastructure that an RL pipeline would require. In that sense, the preliminary system is not merely motivational background. It is the first operational component of the larger compiler architecture.

8 Conclusion

This report explored reinforcement learning as a future direction for adaptive noise-aware quantum compilation. I first developed a baseline experimental framework that compares mapping strategies on simulated NISQ backends using benchmark circuits with known outputs. These experiments show that mapping choice materially affects empirical success probability, and that structured noise-aware heuristics often provide an advantage over random assignment, though the relative ranking of strategies can vary across runs and benchmarks.

Motivated by these findings, I then proposed a full RLCompiler pipeline in which qubit mapping is modeled as a sequential decision process and optimized using reinforcement learning. The proposed design uses a staged reward strategy built from Murali-style proxy objectives, canary circuits, and eventually runtime characterization signals. I argue that this progression is both practical and well justified: it builds from a stable compiler baseline, uses realistic hardware-aware signals, and provides a plausible route toward adaptive policies that improve beyond static heuristics.

The broader significance of this work is that it reframes compilation as a learning problem. In the NISQ era, where noise and hardware variability dominate performance, an adaptive compiler that learns from interaction may be better suited to real devices than one that depends exclusively on fixed handcrafted rules.

Code for the experimental framework and RL environment scaffold is publicly available in the project repository [14].

References

[1] Prakash Murali, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi. 2019. Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1015–1029.

[2] Swamit S. Tannu and Moinuddin K. Qureshi. 2018. A Case for Variability-Aware Policies for NISQ-Era Quantum Computers. *arXiv preprint* arXiv:1805.10224.

[3] Gokul Subramanian Ravi, Jonathan M. Baker, Kenneth N. Smith, Nathan Earnest, Ali Javadi-Abhari, and Frederic T. Chong. 2022. Boosting Quantum Fidelity with an Ordered Diverse Ensemble of Clifford Canary Circuits. *arXiv preprint* arXiv:2209.13732.

[4] Quinn Langfitt, Andres Gonzales, Ji Liu, Zubair H. Saleem, Nikos Hardavellas, and Kenneth N. Smith. 2024. Dynamic Resource Allocation with Quantum Error Detection. *arXiv preprint* arXiv:2408.05565.

[5] John Preskill. 2018. Quantum Computing in the NISQ Era and Beyond. *Quantum* 2, 79.

[6] Gushu Li, Yufei Ding, and Yuan Xie. 2018. Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices. *arXiv preprint* arXiv:1809.02573.

[7] Lorenzo Moro, Matteo G. A. Paris, Matteo Restelli, and Enrico Prati. 2021. Quantum Compiling by Deep Reinforcement Learning. *Communications Physics* 4, 178.

[8] Z. T. Wang et al. 2024. Quantum Compiling with Reinforcement Learning on a Superconducting Processor. *arXiv preprint* arXiv:2406.12195.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint* arXiv:1707.06347.

[10] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22, 1–8.

[11] Richard S. Sutton and Andrew G. Barto. Reinforcement Learning: An Introduction. MIT Press, 2nd edition, 2018.

[12] Qiskit Community. 2017. Qiskit: An Open-Source Framework for Quantum Computing. https://github.com/Qiskit/qiskit.

[13] Farama Foundation. 2023. Gymnasium: A Standard API for Reinforcement Learning. https://github.com/Farama-Foundation/Gymnasium

[14] Ryan Luedtke. 2026. RL-QC: Reinforcement Learning for Adaptive Noise-Aware Quantum Compilation. GitHub repository. https://github.com/ryanluedtke/RL-QC/tree/main